

STREAMIX

Streamix

Technical Architecture & Engineering Reference

TECHNICAL DOCUMENT

Version 1.0 · June 10, 2026

Confidential — For Internal and Investor Use Only

Table of Contents

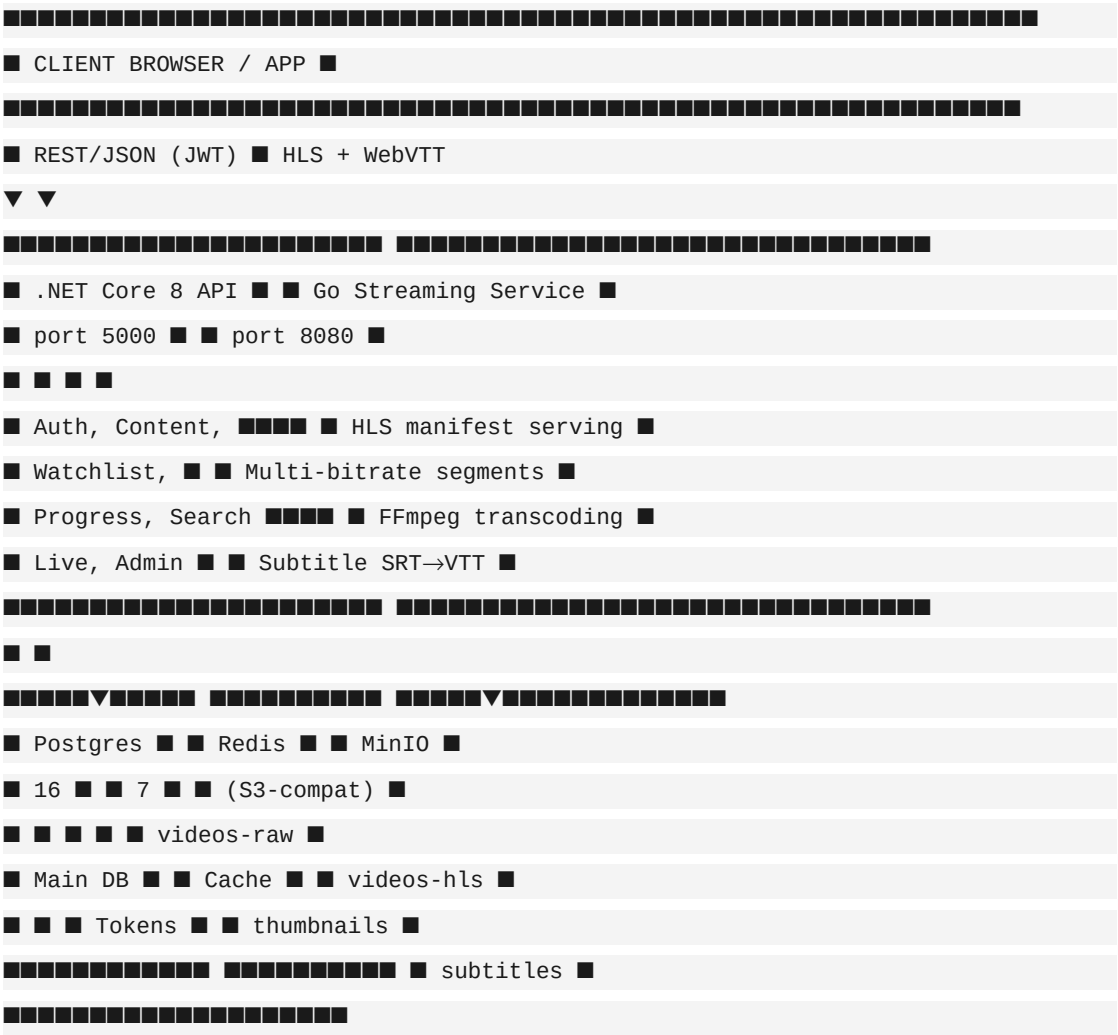
1. System Architecture Overview
2. Technology Stack
3. Repository & Project Structure
4. Backend Service — .NET Core 8
5. Streaming Service — Go
6. Frontend — Next.js 15
7. Database Schema
8. API Reference
9. Infrastructure & DevOps
10. Security Architecture
11. Scalability & Performance
12. Development Workflow

1. System Architecture Overview

Streamix follows a microservices architecture with three independently deployable services, each built in the language best suited to its responsibility. All services are containerised and orchestrated via Docker Compose (development) or Kubernetes (production).

High-Level Architecture

The diagram below shows data flow between client, services, and storage layers:



Service Responsibilities

.NET Core 8 API	Authentication (JWT), user & profile management, content catalogue, watchlist, watch progress, search, subscription management, admin operations, internal callbacks from the Go service. Only service with direct PostgreSQL access.
-----------------	---

Go Streaming Service	Receives raw video paths from the backend, runs FFmpeg to produce multi-bitrate HLS, stores segments in MinIO, and serves signed HLS manifests and segments to clients. No direct DB access — queries backend (Redis-cached) for content metadata.
Next.js 15 Frontend	Server-side rendered and client-side React application. App Router with Server Components for browse/search pages; Client Components for player, watchlist, and interactive UI. Acts as API gateway via Next.js rewrites to avoid browser CORS issues.

2. Technology Stack

Layer	Technology	Version	Rationale
Frontend Framework	Next.js	15.x	App Router, Server Components, built-in rewrites
UI Language	TypeScript	5.x	Type safety across API boundaries
Styling	Tailwind CSS	3.x	Utility-first; dark theme with brand tokens
HLS Player	HLS.js	1.5.x	Adaptive bitrate, quality selection, DRM-ready
Server State	TanStack Query	5.x	Caching, background refetch, infinite scroll
Client State	Zustand	4.x	Auth store with localStorage persistence
Auth (FE)	NextAuth.js	5.x (beta)	App Router compatible; credentials + OAuth
Backend Framework	.NET / ASP.NET Core	8.0 / 10.0	High-performance, mature ecosystem
ORM	Entity Framework Core	8.x	Code-first migrations, LINQ, Npgsql driver
CQRS Bus	MediatR	12.x	Decoupled handlers, pipeline behaviours
Validation	FluentValidation	11.x	Declarative rules, MediatR integration
Mapping	AutoMapper	13.x	Entity ↔ DTO projection
Streaming Lang.	Go	1.24	Goroutines for parallel transcoding; low memory
HTTP Router (Go)	Chi	5.x	Lightweight, idiomatic, middleware-first
Redis Client (Go)	go-redis	9.x	Full Redis 7 support
MinIO Client (Go)	minio-go	7.x	S3-compatible presigned URLs
Transcoder	FFmpeg	6.x	Industry-standard multi-bitrate HLS
Database	PostgreSQL	16	JSONB, tsvector, pg_trgm, uuid_generate_v4
Cache	Redis	7	Strings, sorted sets, TTL-based token storage

Object Storage	MinIO	latest	S3-compatible; self-hosted; no egress fees in dev
Reverse Proxy	Nginx	1.25	SSL termination, routing, static file serving
Containerisation	Docker / Compose	v5	Dev orchestration; prod → Kubernetes

3. Repository & Project Structure

Monorepo layout under **streaming/**:

```
streaming/
├── .env.example
├── docker-compose.yml
├── docker-compose.dev.yml
├── Makefile
├── apps/
│   ├── frontend/ # Next.js 15
│   │   └── src/
│   │       ├── app/ # App Router pages
│   │       ├── components/ # browse/ player/ live/ ui/ layout/
│   │       ├── lib/api/ # axios wrappers per domain
│   │       ├── hooks/ # useDebounce, useWatchlist, useProfile
│   │       ├── store/ # Zustand auth store
│   │       ├── types/ # TS interfaces mirroring backend DTOs
│   │       ├── Dockerfile
│   │       └── package.json
│   ├── backend/ # .NET Core 8
│   │   └── src/
│   │       ├── StreamingPlatform.API/
│   │       ├── StreamingPlatform.Application/
│   │       ├── StreamingPlatform.Domain/
│   │       ├── StreamingPlatform.Infrastructure/
│   │       ├── StreamingPlatform.sln
│   │       ├── Dockerfile
│   │       ├── streaming/ # Go
│   │       ├── cmd/server/main.go
│   │       ├── internal/
│   │       ├── config/ handler/ service/
│   │       ├── storage/ cache/
│   │       ├── transcoder/ middleware/
│   │       ├── go.mod
│   │       ├── Dockerfile
│   │       └── infra/
│   ├── postgres/init.sql # Extensions + schema bootstrap
│   └── redis/redis.conf
```

```
■■■ minio/setup.sh # Bucket creation
```

```
■■■ nginx/nginx.conf
```


4. Backend Service — .NET Core 8

Clean Architecture Layers

Domain	Pure C# entities and enums. No framework dependencies. Entities: User, Profile, Subscription, Content, Season, Episode, VideoAsset, Subtitle, Genre, WatchlistItem, WatchProgress, Channel, LiveEvent.
Application	Use cases implemented as MediatR IRequest command/query handlers. Interfaces (IApplicationDbContext, ITokenService, ICacheService, IStorageService, IStreamingService) defined here; implementations in Infrastructure. FluentValidation validators injected as MediatR pipeline behaviours.
Infrastructure	EF Core ApplicationDbContext, 14 IEntityTypeConfiguration files, TokenService (JWT), CacheService (Redis), MinioStorageService, StreamingServiceClient (HttpClient). All registered via DependencyInjection.cs.
API	7 ASP.NET Core controllers, ExceptionHandlingMiddleware (ProblemDetails RFC 7807), Serilog request logging, Swagger/OpenAPI with Bearer scheme, health checks, auto-migration on startup in development.

Key Domain Enums

Enum	Values
ContentType	Movie Series Documentary ShortFilm
VideoQuality	Q360p Q720p Q1080p Q4K
ProcessingStatus	Pending Processing Ready Failed
SubscriptionTier	Free(0) Basic(1) Standard(2) Premium(3)

JWT Token Design

Algorithm	HS256 with 256-bit secret from environment
Access Token TTL	15 minutes
Refresh Token TTL	7 days, stored in Redis as refresh:{token} → userId
Claims	userId, email, profileId, subscriptionTier, jti
Rotation	Each /auth/refresh issues a new refresh token and invalidates the old one

Search Implementation

PostgreSQL full-text search using a generated **tsvector** column combined with **pg_trgm** similarity for fuzzy matching:

```
-- Generated column on content table:
search_vector tsvector GENERATED ALWAYS AS
(to_tsvector('english', title || ' ' || coalesce(description, ''))) STORED

-- GIN indexes:
CREATE INDEX idx_content_search ON content USING GIN (search_vector);
CREATE INDEX idx_title_trgm ON content USING GIN (title gin_trgm_ops);

-- Query (raw SQL in SearchController):
WHERE similarity(title, @q) > 0.2
OR search_vector @@ plainto_tsquery('english', @q)
ORDER BY similarity(title, @q) DESC, ts_rank(search_vector, ...) DESC
```

5. Streaming Service — Go

Transcoding Pipeline

When the backend receives a confirmed raw video upload it calls **POST /internal/transcode** on the Go service. The service enqueues the job in a bounded worker pool (default: 3 concurrent FFmpeg processes) and returns **202 Accepted** immediately.

```
1. Job received → push to buffered channel (worker pool queue)
2. Worker picks up job → downloads raw file from MinIO (videos-raw bucket)
3. FFmpeg invoked with multi-bitrate filter_complex:
[0:v] split=3 → scale to 360p / 720p / 1080p
libx264 encoding: 800k / 2800k / 5000k bitrates
HLS output: 6-second segments, VOD playlist type
Output: videos-hls/{assetId}/master.m3u8 + v0/ v1/ v2/ segment trees
4. All segment files uploaded to MinIO (videos-hls bucket)
5. PATCH /api/admin/assets/{id}/status called on backend
Body: { status: 'ready', hlsManifestPath, processedAt }
Header: X-Internal-Key: {INTERNAL_API_KEY}
```

HLS Manifest Rewriting (Token Gating)

The Go service does not expose raw MinIO URLs to clients. Instead it rewrites HLS playlist files on-the-fly to replace segment paths with token-gated URLs that pass through the service:

```
Client → GET /stream/{contentId}/master.m3u8?token=JWT
```

```
Go → 1. Validate JWT
```

```
2. Check subscriptionTier >= content.requiredTier
(content metadata from Redis or backend call)
```

```
3. Fetch master.m3u8 from MinIO
```

```
4. Rewrite each variant playlist URL:
```

```
/stream/{contentId}/v{q}/prog_index.m3u8?token=JWT
```

```
5. Return rewritten playlist (200 OK)
```

```
Client → GET /stream/{contentId}/v1/seg003.ts?token=JWT
```

```
Go → 1. Validate JWT
```

```
2. Generate presigned MinIO GET URL (1-hour expiry)
```

```
3. Return 302 redirect to presigned URL
```

```
(MinIO serves bytes; Go does not proxy the stream)
```

Go Project Structure

```
apps/streaming/
■■■ cmd/server/main.go # Wire config, router, start HTTP server
■■■ internal/
■ ■■■ config/config.go # Viper env-based config
■ ■■■ server/server.go # Chi router + CORS + auth middleware
■ ■■■ handler/
■ ■ ■■■ stream_handler.go # Manifest + segment endpoints
■ ■ ■■■ transcode_handler.go # POST /internal/transcode
■ ■ ■■■ health_handler.go
■ ■■■ service/
■ ■ ■■■ stream_service.go # Resolve asset paths, check tiers
■ ■ ■■■ transcode_service.go # FFmpeg orchestration
■ ■■■ storage/minio_client.go # GetObject, PutObject, PresignedURL
■ ■■■ cache/redis_client.go # Token cache, segment metadata
■ ■■■ transcoder/
■ ■ ■■■ ffmpeg.go # Command builder
■ ■ ■■■ pipeline.go # Multi-quality runner (errgroup)
■ ■ ■■■ worker_pool.go # Bounded goroutine pool
■ ■■■ middleware/auth.go # JWT validation (shared secret)
■■■ pkg/models/
■■■ transcode_job.go
■■■ stream_token.go
```

6. Frontend — Next.js 15

App Router Page Structure

Route	Description
/ (root)	Redirects to /browse
/(auth)/login	CredentialsProvider login form with react-hook-form + zod
/(auth)/register	Registration form; calls POST /api/auth/register
/browse	Server Component: Hero + ContentRow per genre (parallel fetch)
/browse/[genre]	Filtered grid; GenreFilter pill navigation
/title/[contentId]	Full detail page: backdrop, synopsis, episodes, watchlist btn
/watch/[contentId]	Full-screen player; no navbar; loads HLS via VideoPlayer
/search	Debounced search bar + TanStack Query result grid
/live	Live Now / Upcoming / TV Channels sections
/live/[eventId]	LivePlayer (low-latency HLS, no seek bar)
/sports	Sport-type filter tabs
/tv	Channel category grid
/profile/select	Who's Watching avatar picker; switches active profile
/profile/[profileId]	Profile settings: name, maturity rating, language
/profile/[profileId]/watchlist	Watchlist grid for active profile
/account	Membership tier, billing history, password change

VideoPlayer Architecture

Library	HLS.js v1.5 with native HLS fallback for Safari
Quality switch	hls.currentLevel = n (-1 = auto); live quality list from hls.levels
Token auth	xhrSetup callback adds Authorization Bearer header to XHR requests
Subtitles	elements added after HLS attach; VTT served from MinIO
Resume	videoRef.currentTime = startPosition after HLS is attached
Keyboard shortcuts	Space/K=play-pause, ←/→=±10s, M=mute, F=fullscreen, ↑/↓=volume

Progress sync	useProgressSync hook: useDebounceCallback(10s) + fire on pause/unmount
Inactivity hide	Controls hidden after 3s of no mouse movement; shown on mousemove

API Client Pattern

All API calls go through **src/lib/api/client.ts** — an axios instance with two interceptors:

- Request interceptor: reads accessToken from Zustand store → sets Authorization: Bearer {token}.
- Response interceptor: on 401 → calls POST /api/auth/refresh with stored refresh token → retries original request; on second 401 → clears store and redirects to /login.

Edge Middleware

src/middleware.ts runs on the Next.js edge runtime and protects routes before any page code executes. Routes /browse/*, /watch/*, /title/*, /profile/*, and /account redirect to /login when no valid session cookie is present.

7. Database Schema

Entity Relationship Summary

Table	Primary Key	Foreign Keys / Relations
users	uuid (uuid_generate_v4)	—
profiles	uuid	user_id → users.id
subscriptions	uuid	user_id → users.id
genres	uuid	—
content	uuid	— (+ generated search_vector tsvector)
content_genres	composite (content_id, genre_id)	both → their tables
seasons	uuid	content_id → content.id
episodes	uuid	season_id → seasons.id
video_assets	uuid	content_id → content.id OR episode_id → episodes.id
subtitles	uuid	content_id / episode_id (nullable)
watchlist	uuid	profile_id → profiles.id, content_id → content.id
watch_progress	uuid	profile_id, content_id, episode_id (nullable)
channels	uuid	— (live TV channels)
live_events	uuid	channel_id → channels.id (nullable)

Critical Indexes

```
-- Full-text search
CREATE INDEX idx_content_search ON content USING GIN (search_vector);
CREATE INDEX idx_title_trgm ON content USING GIN (title gin_trgm_ops);

-- User activity (sort by recency)
CREATE INDEX idx_progress_profile ON watch_progress (profile_id, last_watched_at DESC);
CREATE INDEX idx_watchlist_profile ON watchlist (profile_id, added_at DESC);

-- Content browsing
CREATE INDEX idx_content_type ON content (type, is_published);
CREATE INDEX idx_content_genre ON content_genres (genre_id);

-- Asset lookup
```

```
CREATE INDEX idx_video_asset_content ON video_assets (content_id, quality);
```

```
CREATE INDEX idx_video_asset_episode ON video_assets (episode_id, quality);
```


8. API Reference

Authentication — /api/auth

Method	Path	Auth	Description
POST	/api/auth/register	—	Register new account; returns token pair
POST	/api/auth/login	—	Authenticate; returns token pair
POST	/api/auth/refresh	Refresh	Rotate refresh token; returns new pair
POST	/api/auth/logout	Bearer	Invalidate refresh token in Redis

Content — /api/content

Method	Path	Auth	Description
GET	/api/content	—	Paginated list; ?page&pageSize;&type;
GET	/api/content/featured	—	Featured titles (Redis cached 10 min)
GET	/api/content/trending	—	Trending (Redis cached 10 min)
GET	/api/content/genres	—	All genre slugs + names
GET	/api/content/genre/{slug}	—	Content by genre, paginated
GET	/api/content/{id}	—	Full detail with seasons, assets, subtitles
GET	/api/content/{id}/stream	Bearer	Returns master M3U8 URL + qualities + subtitles
POST	/api/content/{id}/upload	Admin	Returns presigned MinIO PUT URL

Search — /api/search

Method	Path	Auth	Description
GET	/api/search	—	?q=&type;=&genre;=&year;= — fuzzy + full-text search

Watch Progress — /api/progress

Method	Path	Auth	Description
GET	/api/progress	Bearer	All progress for active profile (continue watching)
GET	/api/progress/{contentId}	Bearer	Progress for one title
PUT	/api/progress/{contentId}	Bearer	Upsert position; marks completed at ≥90%

Watchlist — /api/watchlist

Method	Path	Auth	Description
GET	/api/watchlist	Bearer	All watchlist items for active profile
POST	/api/watchlist/{contentId}	Bearer	Add title; idempotent
DELETE	/api/watchlist/{contentId}	Bearer	Remove title

Users & Profiles — /api/users

Method	Path	Auth	Description
GET	/api/users/me	Bearer	Current user info
PUT	/api/users/me	Bearer	Update name / email
GET	/api/users/me/profiles	Bearer	List all profiles
POST	/api/users/me/profiles	Bearer	Create profile (max 5)
PUT	/api/users/me/profiles/{id}	Bearer	Update profile
DELETE	/api/users/me/profiles/{id}	Bearer	Delete profile
GET	/api/users/me/subscription	Bearer	Active subscription details

Streaming Service — Go (port 8080)

Method	Path	Auth	Description
GET	/stream/{contentId}/master.m3u8	?token =	Rewritten HLS variant playlist
GET	/stream/{contentId}/{q}/prog_index.m3u8	?token =	Quality-level HLS playlist
GET	/stream/{contentId}/{q}/{seg}.ts	?token =	302 redirect to presigned MinIO URL
POST	/internal/transcode	X-Key	Enqueue transcode job (backend→Go)
PATCH	/api/admin/assets/{id}/status	X-Key	Transcode complete callback (Go→backend)
GET	/health	—	Liveness probe

9. Infrastructure & DevOps

Docker Compose Services

Service	Image	Port(s)	Depends On
postgres	postgres:16-alpine	5432	—
redis	redis:7-alpine	6379	—
minio	minio/minio:latest	9000, 9001	—
minio-init	minio/mc:latest	—	minio
backend	apps/backend Dockerfile	5000→8080	postgres, redis, minio
streaming	apps/streaming Dockerfile	8080	minio, redis
frontend	apps/frontend Dockerfile	3000	backend, streaming
nginx	infra/nginx Dockerfile	80, 443	frontend, backend, streaming

MinIO Buckets

Bucket	Access	Content
videos-raw	Private	Original uploaded video files (any format)
videos-hls	Public read	Transcoded HLS segments + master playlists
thumbnails	Public read	Poster and backdrop JPEG/WebP images
subtitles	Public read	WebVTT subtitle files per language

Nginx Routing Rules

- location /api/ → proxy_pass http://backend:8080/
- location /stream/ → proxy_pass http://streaming:8080/
- location / → proxy_pass http://frontend:3000/
- client_max_body_size 5G — allows large video uploads through Nginx
- proxy_read_timeout 3600s — prevents timeout during long FFmpeg transcode status polling

Makefile Targets

Target	Command Run
dev	docker compose -f docker-compose.yml -f docker-compose.dev.yml up
build	docker compose build

migrate	<code>docker compose run --rm backend dotnet ef database update</code>
seed	<code>docker compose run --rm backend dotnet run --project tools/Seeder</code>
test-backend	<code>cd apps/backend && dotnet test</code>
test-streaming	<code>cd apps/streaming && go test ./...</code>
test-frontend	<code>cd apps/frontend && pnpm test</code>
lint-go	<code>cd apps/streaming && golangci-lint run</code>

10. Security Architecture

Authentication & Authorisation

- Passwords hashed with bcrypt (cost factor 12). Never stored in plaintext.
- Access tokens are short-lived (15 min) to limit blast radius of token theft.
- Refresh tokens are opaque random strings stored server-side in Redis. Stolen refresh tokens can be revoked instantly by deleting the Redis key.
- Refresh tokens are rotated on every use — each refresh invalidates the previous token.
- Subscription tier is embedded in the JWT claim; backend and Go service both enforce access.
- Profile switching requires a separate claim update (new JWT issued); PIN-protected profiles require PIN verification before JWT is re-issued.

Transport Security

- All external traffic via HTTPS (Nginx terminates TLS; cert from Let's Encrypt in production).
- Internal Docker network: services communicate over the Docker bridge network, not exposed externally.
- Internal API calls (backend ↔ Go) authenticated with X-Internal-Key shared secret.
- HLS segments gated by short-lived JWT query tokens; even public-read MinIO URLs are presigned with 1-hour expiry.

Input Validation & Injection Prevention

- All backend inputs validated by FluentValidation before reaching handlers.
- EF Core uses parameterised queries throughout — no raw string concatenation.
- Raw SQL in SearchController uses Npgsql parameters (@q) — immune to SQL injection.
- Frontend form inputs validated with zod schemas before submission.
- File upload restricted to video MIME types; file size validated server-side.

Planned Security Enhancements (Phase 2)

- Widevine (Chrome/Android) + FairPlay (Safari/iOS) DRM for Premium tier content.
- Rate limiting on auth endpoints (Redis sliding window, 5 attempts / 15 min).
- OWASP-aligned Content Security Policy headers via Nginx.
- Regular dependency audit: dependabot for NuGet/npm/Go; automated CVE scanning in CI.

11. Scalability & Performance

Horizontal Scaling Strategy

Frontend (Next.js)	Stateless — scale replicas behind Nginx load balancer. SSR pages cached at CDN edge (Cloudflare) with stale-while-revalidate.
Backend (.NET Core)	Stateless — session data in Redis, no in-memory state. Scale to N replicas; PostgreSQL connection pool via Npgsql (default: 10–100 connections). Read-heavy endpoints backed by Redis (10-min TTL for featured/trending content).
Go Streaming	Worker pool controls FFmpeg concurrency (default: 3). HLS serving is cheap — just presigned URL redirects, no byte proxying. Scale replicas freely; MinIO is the shared state.
PostgreSQL	Read replicas for read-heavy browse/search traffic. Write traffic (progress updates, watchlist) to primary only. GIN indexes keep search fast at 1M+ content rows.
Redis	Redis Cluster for high availability. Separate logical databases for tokens (TTL-critical) vs. content cache.
MinIO / CDN	MinIO in production replaced by or fronted by S3 + CloudFront. HLS segments served at edge — each .ts file is ~200KB, highly cacheable.

Performance Targets

Metric	Target
Time to first byte (API)	< 100ms p95
Search response time	< 200ms p95 (PostgreSQL GIN)
HLS manifest response	< 50ms (Redis-cached path resolution)
Transcode time (90-min movie)	< 25 min (3 quality levels parallel)
Segment cache hit rate	> 85% (CloudFront / Nginx proxy cache)
Player startup time	< 3 seconds on 5 Mbps connection
Uptime SLA	99.9% monthly

12. Development Workflow

Local Development Setup

```
1. git clone https://github.com/remon024/streミング && cd streミング
2. cp .env.example .env # fill in secrets
3. make dev # starts all 7 Docker containers
4. make migrate # runs EF Core migrations
5. make seed # loads sample genres + content
6. Open http://localhost:3000 (frontend)
Open http://localhost:5000/swagger (backend API docs)
Open http://localhost:9001 (MinIO console)
```

End-to-End Upload & Stream Test

```
1. Register admin account: POST /api/auth/register
2. Get presigned upload URL: POST /api/content/{id}/upload
3. Upload video file via HTTP PUT to presigned URL
4. Backend calls POST http://streaming:8080/internal/transcode
5. Go service transcodes → uploads HLS → calls PATCH
  /api/admin/assets/{id}/status
6. Poll GET /api/content/{id} until videoAssets[].status === 'ready'
7. GET /api/content/{id}/stream → returns masterM3u8Url
8. Open /watch/{id} in browser → VideoPlayer loads master.m3u8
9. Confirm 360p / 720p / 1080p quality levels appear in QualitySelector
10. Play 30 seconds → check PUT /api/progress/{id} fires
11. Refresh page → confirm player resumes at correct position
```

Branch Strategy

main	Production-ready code only. Protected branch.
develop	Integration branch for feature PRs.
feature/*	Individual feature branches off develop.
hotfix/*	Critical fixes branched from main; merged to both main and develop.

Testing Strategy

- Backend: xUnit unit tests for Application handlers + Infrastructure services. Integration tests use TestContainers (real PostgreSQL + Redis containers).

- Go: Standard library testing package; table-driven unit tests for FFmpeg command builder and HLS manifest rewriter.
- Frontend: Vitest + React Testing Library for component tests; Playwright for E2E.
- CI pipeline (GitHub Actions): lint → unit tests → integration tests → Docker build on every PR.

— End of Document —